

Maestro Cloud Control Architecture Overview

Developer's Guide

M3DG-18

December 2025

Version .1.1

Contents

PREFACE	3
ABOUT THIS GUIDE	3
AUDIENCE	3
THE STRUCTURE OF THE GUIDE	3
1 INTRODUCTION TO MAESTRO SERVICES	4
2 MAESTRO SERVICES MAIN REQUIREMENTS	5
3 SERVICES CORE FEATURES	6
3.1 AUTHENTICATION	6
3.1.1 <i>Web Service Authentication</i>	6
3.1.2 <i>Dynamic CLI Authentication</i>	7
3.2 AUTHORIZATION	7
3.3 ORGANIZATION STRUCTURE MANAGEMENT	8
3.4 MODULAR APPROACH	9
3.4.1 <i>Modular Approach Overview</i>	9
3.4.2 <i>Data Layer Integration</i>	10
3.5 HEALTH CHECK	11
3.6 AVAILABILITY MONITORING/TRACING	12
3.7 ASYNC JOBS TRACKING	12
4 FEATURE TEMPLATES	14
4.1 EVENT DRIVEN JOBS	14
4.2 ETL PIPELINE	15
4.3 SCHEDULED JOBS	17
TABLE OF FIGURES	18
VERSION HISTORY	19

PREFACE

ABOUT THIS GUIDE

The guide describes the main architecture components, approaches and tools, used for developing Maestro Services.

AUDIENCE

The guide is targeted on the developers and architects, planning to develop and integrate new Maestro services.

THE STRUCTURE OF THE GUIDE

The guide consists of the following sections:

1. [Introduction to Maestro Services](#) – provides the overview of Maestro and the services platform concept.
2. [Maestro Services Main Requirements](#) – provides the overview of the features and templates required for Maestro Services.
3. [Services Core Features](#) – provides the detailed information on the core features required for Maestro Services.
4. [Feature Templates](#) – provides the detailed information on the patterns and flows that are to be used in Maestro Services.

1 INTRODUCTION TO MAESTRO SERVICES

Maestro Framework allows unified centralized access to hybrid and multi-cloud infrastructures in public clouds and private datacenters.

Maestro's API-first approach enables high flexibility to the solution and allows tech teams quickly integrate new components.

Within Maestro evolution, the core application acquired a wide set of services, meeting different types of enterprise needs. These are services related not only to infrastructure management, but also to billing collection and reporting, security and compliance, automation, etc.

These services can successfully cover the operation and infrastructure tasks on the enterprise level. They can be gathered into a single platform, developed, managed and supported by a dedicated team.

Meanwhile, the enterprise applications facing the business tasks, get more space and capacity. Their teams can perform more effectively, as they do not need to split attention between business and operations.

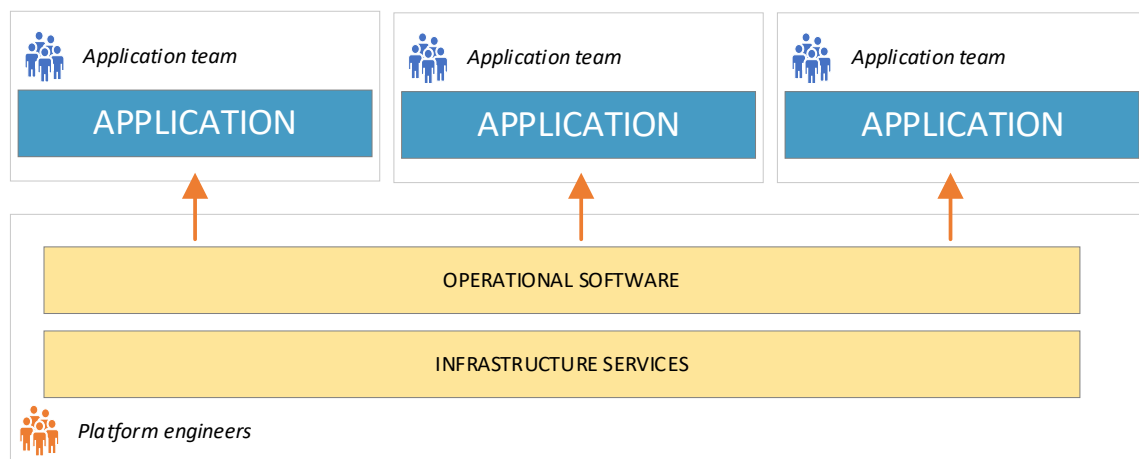


Figure 1 - Generalized platform approach



Maestro services can be used both via Maestro application and MCDM (Maestro common domain model) – an SDK that provides integration between services in the ecosystem on the data layer.

For a service to perform properly, it needs to be built according to the generalized architecture and principles, described further in this document.

2 MAESTRO SERVICES MAIN REQUIREMENTS

Each Maestro Service, irrespective of its purpose and used tools, should include a set of the standard elements – core features and feature templates.

Core Features are the components of the system that each service should include:

- **Modular architecture:** the service should have its own compute capacity and DB and be an independent entity.
- **Authentication/Authorization:** the service should provide properly configured authentication/authorization functionality.
- **Organization structure management:** the service should “understand” the enterprise organization structure and automatically detect any changes in it.
- **Health check:** the service should include self-check functionality and have the possibility to provide brief and detailed report for the support team via a common dashboard.
- **Availability monitoring/Tracing:** the service should enable understanding of the availability, SLA, and downtimes.
- **Async job tracking:** the service should provide understanding of the internal actions status, and notify the client if everything works as expected.

Feature Templates are the patterns that are used within the services performance flows.

- **Event driven jobs:** the service should react on events within other systems according to its business value. For example, in a continuous compliance monitoring service, a change initiates a vulnerability scan with further informing on the target elements security status.
- **ETL Pipeline:** the service should provide proper data aggregation for further processing and reporting.
- **Scheduled jobs:** the service should manage scheduled jobs in a proper manner.



The Open Cloud Management Platform and MCDM are Python-based solutions initially hosted in AWS.

Most service components can be run in container environment (any cloud). However, some elements require using AWS only (this is specified in a specific element description)

3 SERVICES CORE FEATURES

Core Features are the components of the system that each service should include:

- **Modular architecture:** the service should have its own compute capacity and DB and be an independent entity.
- **Authentication/Authorization:** the service should provide properly configured authentication/authorization functionality.
- **Organization structure management:** the service should “understand” the enterprise organization structure and automatically detect any changes in it.
- **Health check:** the service should include self-check functionality and have the possibility to provide brief and detailed report for the support team via a common dashboard.
- **Availability monitoring/Tracing:** the service should enable understanding of the availability, SLA, and downtimes.
- **Async job tracking:** the service should provide understanding of the internal actions status, and notify the client if everything works as expected.

3.1 AUTHENTICATION

The service should use one or both available authentication methods, depending on the client type: Web Service and Dynamic CLI one.

3.1.1 Web Service Authentication

Web services authentication uses common approach that engages AWS services: API gateway, Lambda, and Cognito.

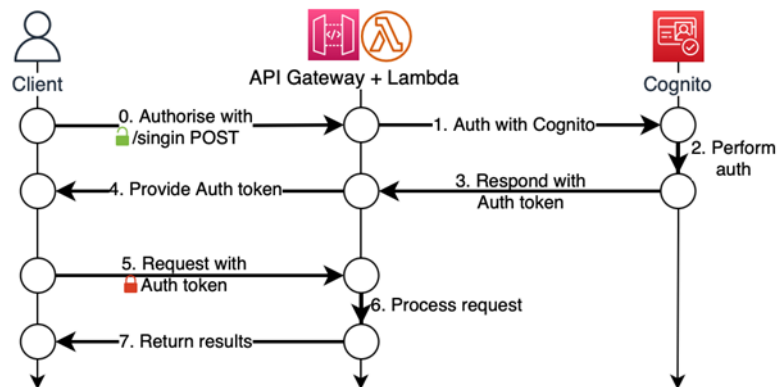


Figure 2 - Web Service Authentication

There is an open sign in post to where the client provides its keys. These keys are authorized with Cognito, which returns a temporary token. Each request is signed with this token, which is checked by AWS out of the box.

3.1.2 Dynamic CLI Authentication

The Dynamic CLI Authentication flow is identical to that for Web service.

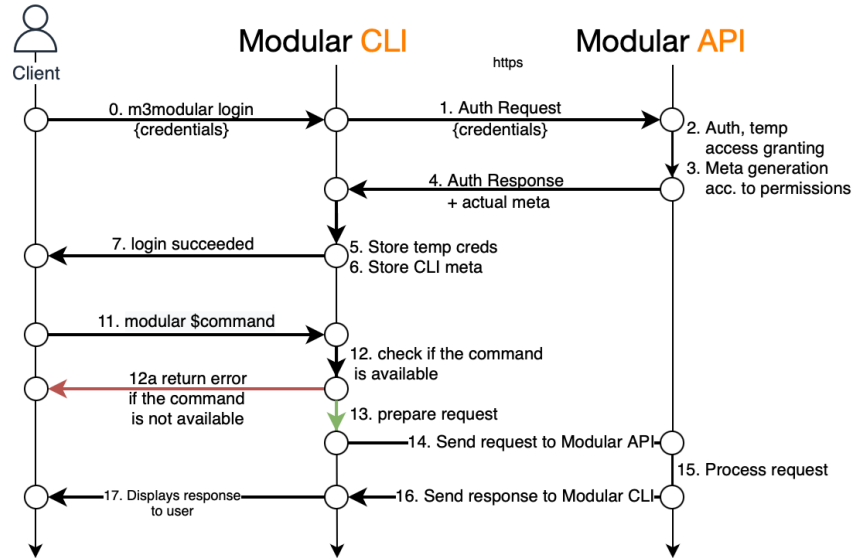


Figure 3 - Dynamic CLI Authentication

The difference is that a custom JWT implementation is used. Also, together with the authentication request, if it is approved, the Modular API returns the content, by which the interface is built (CLI but API is possible), to the thin client (Modular CLI).

Thus, the client will be able to access and communicate only with endpoints available by its permissions. This directly affects which methods will be available to the end users. Once their credentials are updated, the available features on UI will also be updated.

3.2 AUTHORIZATION

A simplified Attribute-Based Access Control (ABAC) is used for authorization.

As opposed to, RBAC that where the access is based on a single attribute – role, ABAC allows better granularity. It allows to provide access on the level of a specific entity attribute or action.



Figure 4 - ABAC Authorization

The allowance is specified on a level of a specific resource, for example a job. For example, one user can create, run describe, delete job. And the other can only describe.

The “simplified ABAC” means that the model is hybrid, has less entities than in a standard ABAC and is very alike to what AWS IAM provides.

Maestro Services ABAC operates with the following items:

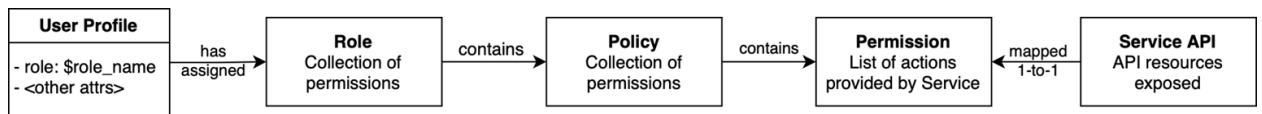


Figure 5 - ABAC Structure

- A user profile.
- The user profile always has an assigned role.
- A role includes policies.
- A policy contains the permissions describing what a user can do with specific entities. This can be explicit list of permissions, or bulk specification via wildcards.



Each API point and method is mapped to one permission.

3.3 ORGANIZATION STRUCTURE MANAGEMENT

Maestro Services should support the following elements within an organization:

- **Customer:** represents a Customer's Organization. It is a container entity for Tenants.
One Customer may have many Tenants.
- **Tenant:** represents a subunit of the Customer, which works directly with an AWS/Azure/GCP account.
In fact, Tenant is a representation of cloud account.
One Tenant belongs only to a single Customer.
(In Maestro system, it is mapped per AWS/AZUR/GCP account, but can have a whole block of accounts).
- **Parent:** describes a specific service availability for a specific Tenant. Includes the detailed availability configuration (for example, the set of service features, available for a specific tenant).
A Parent is associated with a Tenant via Tenant ID, and uses an Application ID to address a specific service.
A Parent may be used as a configuration for some feature module available for Tenant without having an Application associated.
- **Application:** represents a third-party service integration available within the Ecosystem. An Application has the following attributes:
 - id
 - type
 - type specific ones
 - linked application id (#optional)

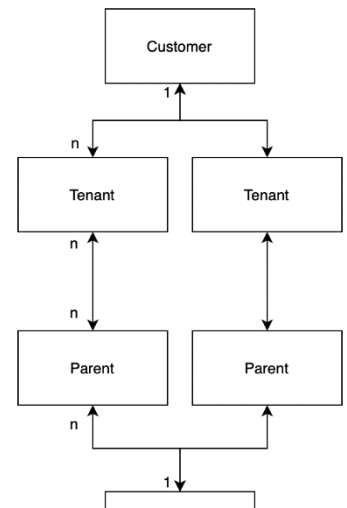


Figure 6 - Organization Structure

An existing Application means there is an service integration which may be activated for a whole Customer or some Tenants within the Customer.

The Application object may have the 'linked application id' which stores an ID of another Application, used by the service logic.

3.4 MODULAR APPROACH

Having numerous services, each with its own specific API leads to operational and support overload within the team responsible for centralized support of such services.

That is why we developed a modular façade which allows to combine different services controls under one custom API/CLI/UI service.

3.4.1 Modular Approach Overview

The Modular approach is close to that of the Microservice architecture, up to the point where each service has its own compute and DB.

A service is actually a module. It includes a Compute part (Lambda, EC2, EKS, AWS butch), it has an API (any), and this API has a CLI client. The CLI client is a descriptor for modular façade.

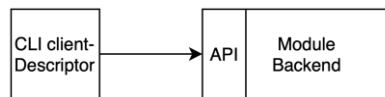


Figure 7 - Module/Service Scheme

Typically, a service uses the following elements:

- **API:** API Gateway, Lambda Function URL, Container exposed, AMQP
- **Authentication:** Custom JWT / Cognito
- **Authorization:** ABAC
- **CLI:** Thin client, descriptor for Modular Facade
- **Compute:** Lambda, AWS Batch, EKS
- **Runtimes:** Python 3.8+, Java 11, NodeJS 18.16.0
- **Persistence:** AWS DynamoDB, MongoDB Atlas, AWS DocumentDB, AWS RDS, S3

The Modular façade is built as described on the picture below.

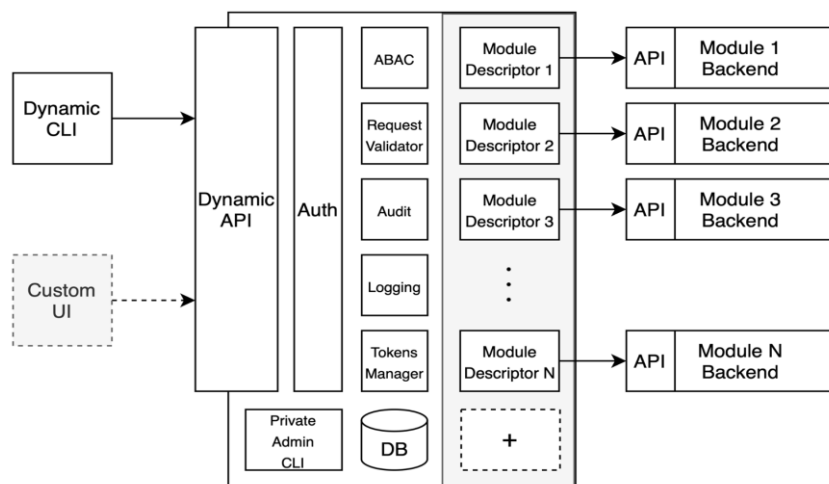


Figure 8 - Modular Facade

The Modular Façade includes interface descriptors (CLI Clients). A descriptor is a file which specifies an entry point to the module. The Click Python library is used to describe the interface. This information is parsed into a specific structure in the modular façade.

After that, the Façade puts its own feature set atop of the API/model/response that:

- Provides its own unified authentication mechanism.
- Provides its own unified ABAC. This allows to have multiple services managed in a unified way from a single entry point and with the same permissions settings.
- Provides pre-validation of requests before the request is sent to the Module backend.
- Provides centralized audit, logging, and manages the temporary tokens retrieved during the client authentication on module backends.
- Has its own database and a CLI that is not available from outside.

Modular façade can get potentially endless number of services, but the number will in fact be limited by the cognitive complexity of management of the whole system (the estimated capacity, from this point of view, is 20-30 services).

API provided by the Modular Façade is generated based on the installed modules. Dynamic API gets the methods based on the modules.

3.4.2 Data Layer Integration

Each service needs data for performance. The data can be logically divided into service specific and common ones. Allowing all services address common tables at once would lead to performance issues of the whole ecosystem.

Thus, to enable comfortable working with data, MCDM (Maestro common domain model) was introduced.

MCDM is an SDK that provides integration between services in the ecosystem on the data layer. implements all general classes, mechanisms and algorithms of work with the common tables/persistence.

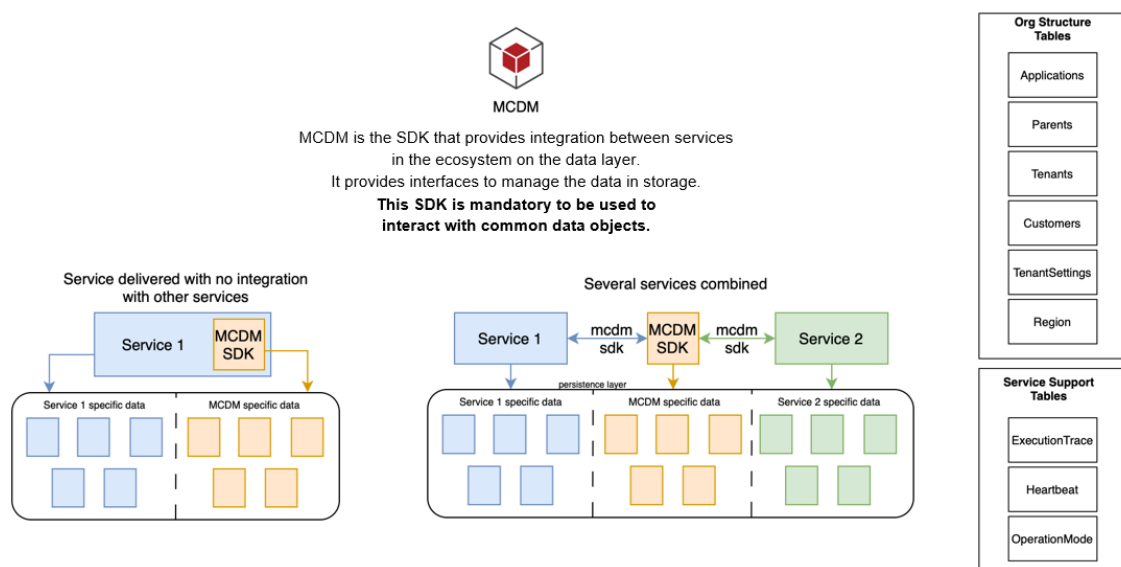


Figure 9 - Data layer integration with MCDM

A service should work only with its own tables and the tables within the MCDM, using MCDM SDK component. Thus, several services can have the same version of MCDM SDK and use the MCDM tables that describe the organization structure or are addressed within the service support flows.

Once a new unit, or tenant appears within the organization, all services describe the change immediately and will be able to access this unit.

3.5 HEALTH CHECK

Each service should include a self-health check mechanism, configured to detect issues in performance and notify the centralized health dashboard on the status.

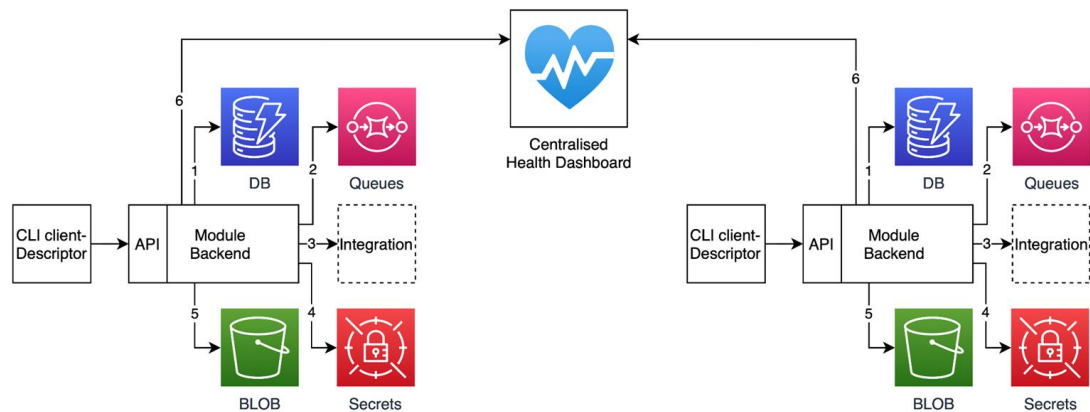


Figure 10 - Service health check

The self-check mechanism should:

1. Check configuration & data consistency
2. Check all queues configured as expected
3. Check integrated services are available
4. Check all secrets exist
5. Check all blob storages exist & configured as expected
6. Push health check report

3.6 AVAILABILITY MONITORING/TRACING

The mechanism is necessary to understand that a service performs as expected, without addressing its API. It analyzes API usage, performance time, error rates and other indicators.

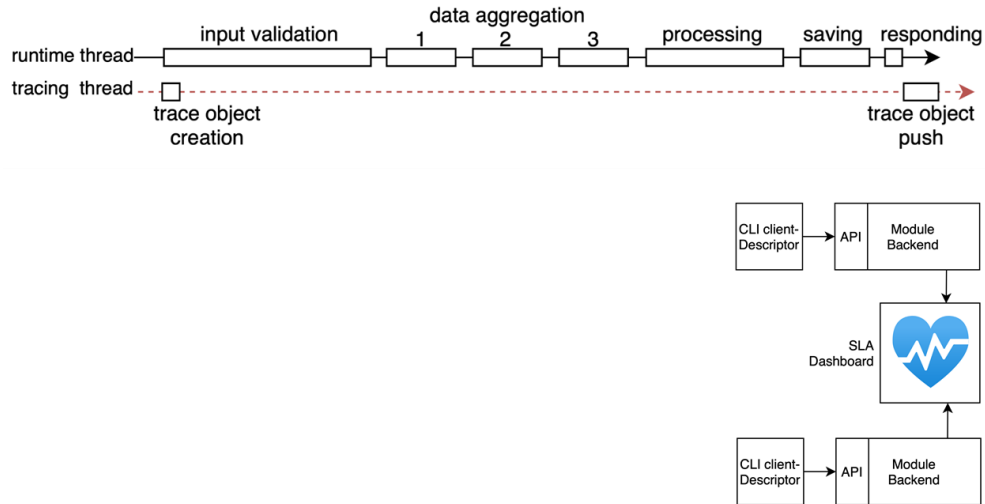


Figure 11 - Availability Monitoring/Tracing thread

There is the main runtime thread with the typical web service flow.

Tracing thread, in it turns, detects that a task execution started, fixes the time and creates a trace object. After the runtime thread finishes, the tracing receives the information about the task performance and the results (status – success or error). The data is written to the trace object, and the object is sent to the SLA dashboard.

Tracing does not affect the time needed for the request processing and retrieving the response. However, the overall run time of the process increases due to the trace object push action.

3.7 ASYNC JOBS TRACKING

Jobs tracking is established in a way similar to the tracing. There are two threads – runtime one, and jobs monitoring one.

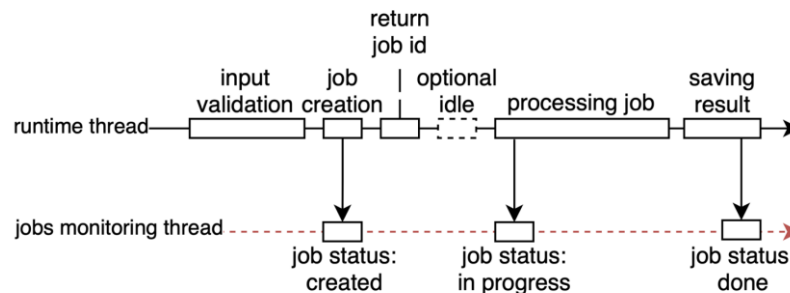


Figure 12 - Async Job Tracking

The jobs monitoring thread receives information about the job status once it changes, and at the end receives the final result. In case an error occurs, the details can be received by Job ID

Job processing flow:

- The client gets status on specific job execution stages immediately.
- A synchronous job validation is performed. If job validation passed successfully, a Job ID is returned. Otherwise, the client gets a error with the details.
- If the validation passed successfully, the service goes on with the processing of the request.
- The client addresses the service for the result of the job execution, and gets all the details (by default, for the last job execution).

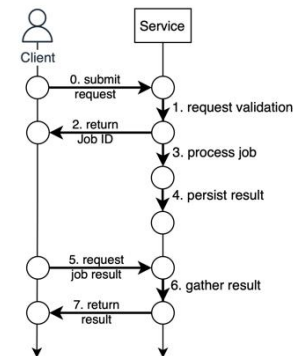


Figure 13 - Job processing flow

4 FEATURE TEMPLATES

Feature Templates are the patterns that are used within the services performance flows.

- **Event driven jobs:** the service should react on events within other systems according to its business value. For example, in a continuous compliance monitoring service, a change initiates a vulnerability scan with further informing on the target elements security status.
- **ETL Pipeline:** the service should provide proper data aggregation for further processing and reporting.
- **Scheduled jobs:** the service should manage scheduled jobs in a proper manner.

4.1 EVENT DRIVEN JOBS

Event driven jobs enable reaction to the changes in external environments/systems. The current implementation is AWS-only, but it can be adopted for any cloud.

A highly-available Event Driven processor used for Maestro services includes:

- 2 or 3-steps processing
- Events buffering

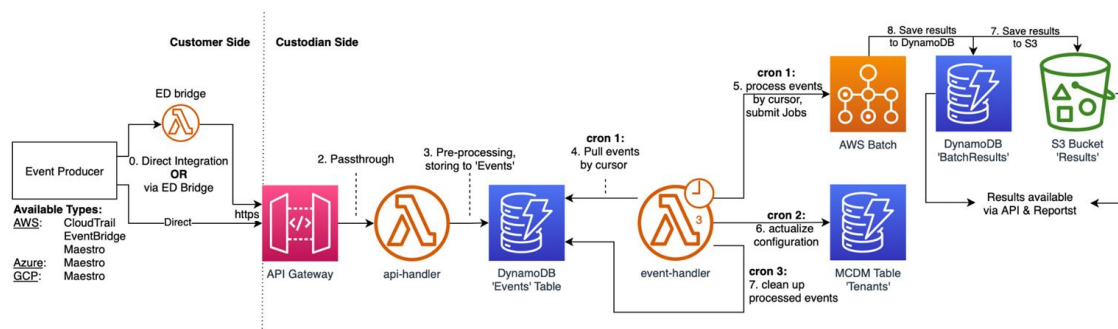


Figure 14 - Event-Driven Job Scheme

Due to ED Bridge usage, the input model converter can be extended to any data model that allows to pass the expected model for the event driven processor.

With Maestro, both native (for AWS) and Maestro custom models (for Azure and GCP) are used.

There are two possible integration flows:

- **Via Event-Driven bridge** – a component that will describe the events regularly according to the client's needs.
- **Receiving events via API** from the producer.

Once the event is received, it is processed according to the following flow:

1. Authentication/Authorization
2. All requests as a batch are passed through Lambda
3. Pre-processing, filtering and saving to Dynamo DB for further processing.
4. A cron-activated processor retrieves the events, prepares them for processing.
5. Events handling:

- 5.1. AWS Batch processes the events, and saves the results to DynamoDB and S3. After that, the results are available via API and in reports.
- 5.2. Use MCDM to actualize the configuration and events table.

4.2 ETL PIPELINE

ETL (extract, transform load) is a data storing process, when data from different sources are retrieved, transformed into a single format and stored for further usage (in a single or multiple storages).

The resulting data can be used to produce analytics, build reports, display on the UI, etc.

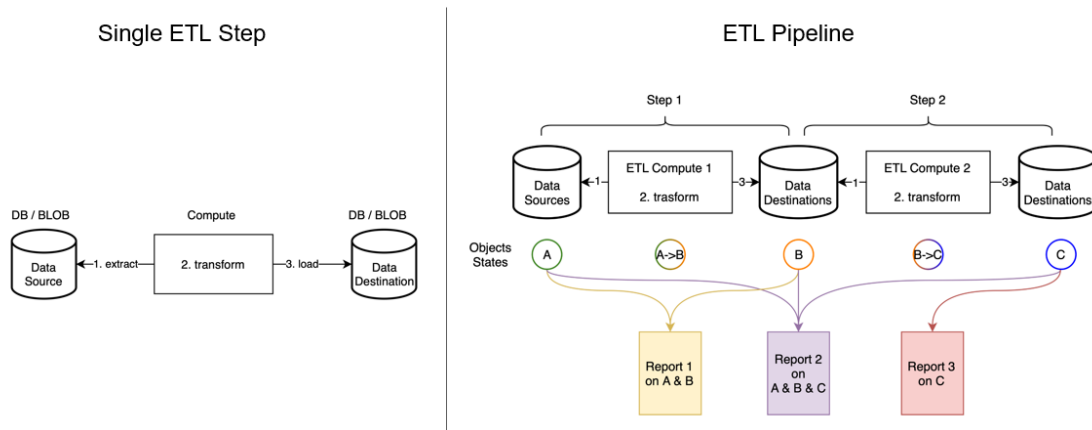


Figure 15 - ETL concept and Maestro Services Pipeline

The architecture introduces ETL Pipeline as an approach where:

1. Data is retrieved from multiple data sources and stored.
2. Data is taken from the storage and transformed.
3. During the process, the specific objects are transformed and stored to the specific destinations. Some objects can be transformed for several times, depending on the final goal.
4. Reports are built based on the different object states.

The example below illustrates an ETL pipeline setup for the continuous compliance monitoring service.

As compute elements, AWS Lambda is used. As storage, AWS S3 and Dynamo DB are used.

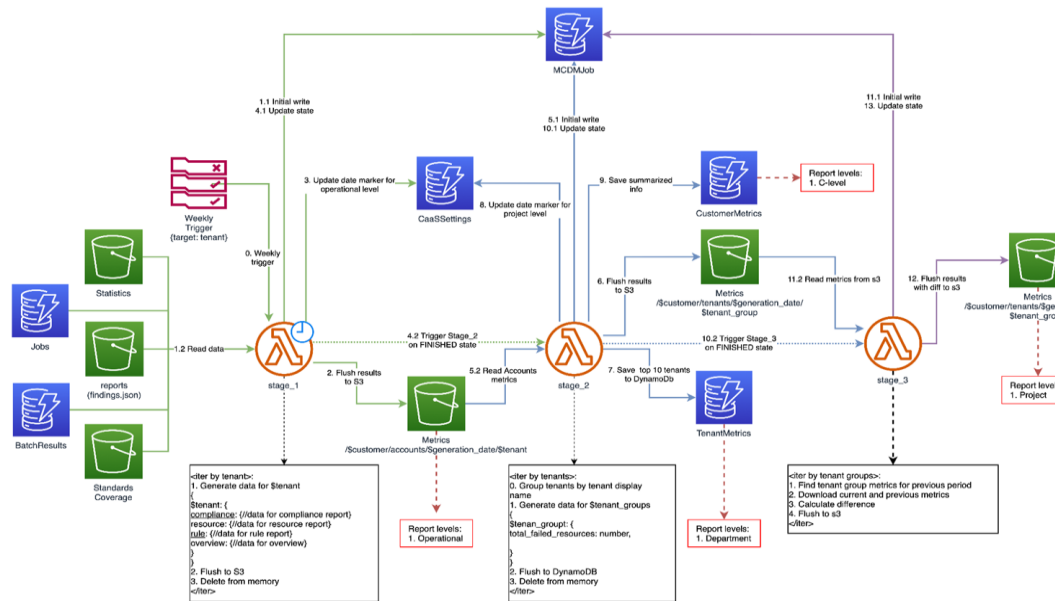


Figure 16 - ETL Pipeline for Continuous Compliance Monitoring Service

The resulting information is delivered in 18 types of reports, all displaying the current information on different levels of organization structure.

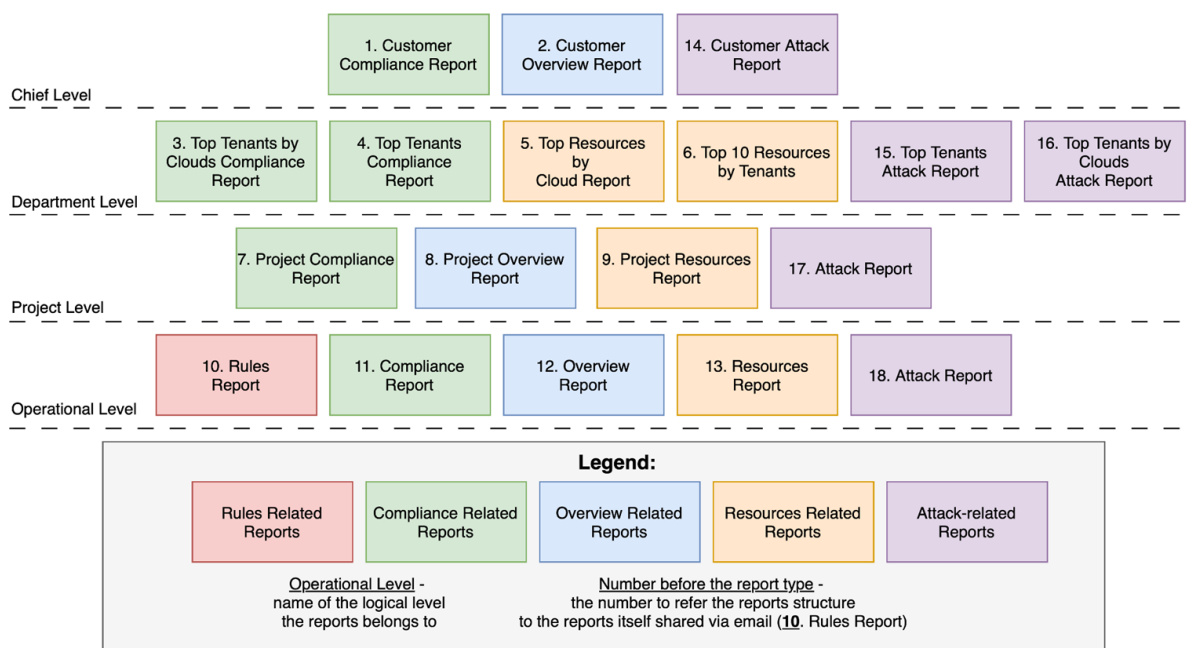


Figure 17 - Continuous Compliance Monitoring reports retrieved as the results of the respective ETL pipeline

4.3 SCHEDULED JOBS

To process scheduled jobs, an external service is used. For AWS, this is AWS EventBridge.

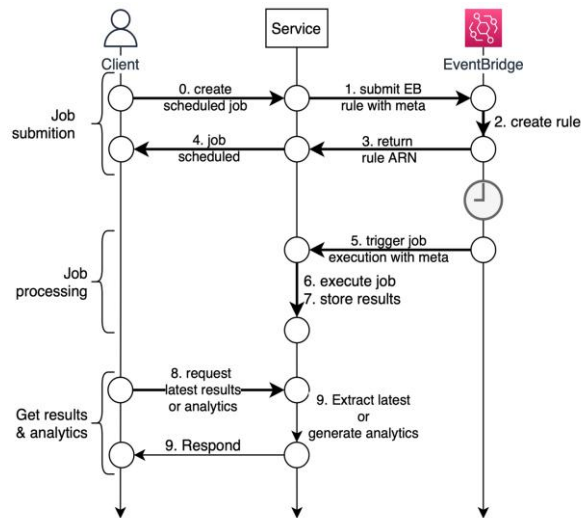


Figure 18 - Jobs Scheduling

Once a job is scheduled, an EventBridge rule is created. It triggers a user-specific API that is defined in the rule meta.

After the job is executed, the client receives both result and analytics on the job.

TABLE OF FIGURES

Figure 1 - Generalized platform approach	4
Figure 2 - Web Service Authentication	6
Figure 3 - Dynamic CLI Authentication	7
Figure 4 - ABAC Authorization	7
Figure 5 - ABAC Structure	8
Figure 6 - Organization Structure	8
Figure 7 - Module/Service Scheme	9
Figure 8 - Modular Facade	9
Figure 9 - Data layer integration with MCDM	10
Figure 10 - Service health check	11
Figure 11 - Availability Monitoring/Tracing thread	12
Figure 12 - Async Job Tracking	12
Figure 13 - Job processing flow	13
Figure 14 - Event-Driven Job Scheme	14
Figure 15 - ETL concept and Maestro Services Pipeline	15
Figure 16 - ETL Pipeline for Continuous Compliance Monitoring Service	16
Figure 17 - Continuous Compliance Monitoring reports retrieved as the results of the respective ETL pipeline	16
Figure 18 - Jobs Scheduling	17

VERSION HISTORY

Version	Date	Summary
1,1	December 12, 2025	Reviewed
1.0	Apri 07l, 2023	The initial version created